

QuickRuit — Product Requirements Document (PRD)

Covers the full employer + candidate flow as described, current build state, target state, and an honest feasibility assessment for every open item — written for a 3-month B2B (HR-tech / staffing agency) build window.

0. Vibe Check — The Short Answer First

Most of what you described is **genuinely buildable in 3 months**, but a few pieces are not fully in your control (they depend on other companies' systems, not just your engineering) and one piece (screen-share detection) has a hard technical ceiling regardless of effort. Breakdown:

- **Fully feasible, straightforward engineering:** job creation with module/duration selection, magic link + DPDPA consent + resume upload flow, candidate approval, interview gating/progression logic, coding sandbox (DSA + take-home), unified scorecard, tab-switch/copy-paste/face-presence proctoring.
- **Feasible but needs real engineering investment, not just a feature toggle:** dynamic AI interviewer ("Recruiter Rian"), question bank at scale, latency-optimized real-time conversation.
- **Feasible but depends on external parties, not just your code:** ATS integration (each ATS is a different partnership/API), job board push (LinkedIn/Naukri gate this behind enterprise partnerships).
- **Not realistically feasible as originally framed, needs re-scoping:** auto-pushing jobs directly onto an arbitrary company's own career page (e.g., "Amazon's career page") — there is no generic way to write into another company's website; this needs to be re-scoped into something you actually control (see Section 4.3).
- **Technically unreliable, set expectations honestly:** detecting whether a candidate is screen-sharing to another device/app outside the browser. Browsers do not expose this reliably. You can detect tab/window blur and multiple displays, not "is Zoom running and sharing this screen to someone."

Full reasoning for every item is in Section 5.

1. Product Context

Goal for this build window (3 months): a B2B product for HR teams and staffing agencies to create a job, define an evaluation (aptitude / technical assessment / AI video interview / coding), invite candidates via a link, have candidates go through a gated, proctored evaluation, and give the employer one unified, evidence-backed scorecard per candidate.

Primary focus for this phase: technical roles. Non-technical role support is a stated future direction, not a day-1 requirement.

2. Employer Flow (Current + Target)

1. **Onboarding & verification** — employer signs up, gets verified, lands in the employer dashboard. *(Built.)*

2. **Job creation** —
 - Job description + requirements (skills, e.g. React, Next.js)
 - Module selection: Aptitude / Technical Assessment / AI Video Interview / Coding Assessment (DSA and/or take-home "machine round")
 - Duration per module or per whole flow (15 min / 30 min blocks)
 - Location, salary, other requirements (*Built — this is the strongest, most complete part of the current flow.*)
 3. **Publish job** — generates a shareable "magic link." (*Built.*)
 4. **Distribution (target, not fully built):**
 - Push/share to job boards (LinkedIn, Naukri) — *see 4.1, needs re-scoping*
 - Auto-publish to the employer's own career page — *see 4.3, needs re-scoping*
 - ATS sync (push published job + pull candidate data back) — *see 4.2*
 5. **Candidate review** — employer sees applicants, views profile + LinkedIn link in one click, approves candidates for interview. (*Built.*)
 6. **Interview invite** — sent to the candidate's verified email once approved. (*Built.*)
 7. **Scorecard review (target state)** — employer sees one unified, evidence-backed scorecard per candidate covering every module they selected, plus the ranked shortlist. (*Partially built — currently only the AI video interview module produces a full scorecard; other modules only show raw right/wrong marks. This is the single most valuable near-term fix — see Section 6.*)
-

3. Candidate Flow (Current + Target)

1. Candidate clicks the magic link (from job board, career page, or direct share) → lands on public job/career page.
2. Accepts DPDPA consent → applies.
3. Verifies email.
4. Uploads resume → application submitted.
5. (After employer approval) receives interview invite on verified email, clicks link.
6. **System security/proctoring check page** — camera/mic check, environment check, consent to monitoring (tab-switch detection, copy-paste blocking, face presence/analysis). (*Largely built, per existing product docs — MediaPipe-based.*)
7. **Target addition — Start Interview landing page:** before the interview begins, candidate sees a clear breakdown of which modules they'll go through and how long each will take, so there's no surprise mid-interview. (*Not yet built — straightforward to add.*)
8. **AI interviewer introduction ("Recruiter Rian")** — introduces itself, explains the flow will begin with whichever module comes first, guides the candidate module-to-module conversationally, gives light encouragement/time-remaining nudges between stages. (*Not yet built — see Section 5 for feasibility.*)
9. **Module sequence** (order should be configurable per job, described sequence was Aptitude → Technical Assessment → Coding → AI Video Interview, but you also described coding-before-video as an option — **decide one canonical default order and make it configurable**, don't leave the order ambiguous in the actual product):
 - **Aptitude round** — gated: candidate cannot proceed until time is up or the round is complete.
 - **Technical assessment round** — same gating logic.
 - **Coding assessment** (DSA and/or take-home/"machine round") — candidate can ask for help; AI should give hints, never answers, and should give light encouragement, not solutions.
 - **AI video interview** — dynamically generated questions based on JD + resume match, spoken/recorded answers.
10. **Exit** — candidate finishes, session ends.
11. **Employer sees the unified scorecard** — target state, see Section 6.

4. Distribution & Integration — Feasibility Deep-Dive

4.1 Job board push (LinkedIn, Naukri)

Reality: LinkedIn and Naukri do not offer open, self-serve APIs for arbitrary third parties to auto-post jobs. LinkedIn's job-posting API access requires becoming an approved Talent Solutions partner (a formal partnership process, not just API keys you can request and get same-day). Naukri has similar enterprise-only integration paths.

What's actually feasible now: generate a clean, shareable public job page + magic link that a recruiter manually pastes into a LinkedIn/Naukri post. This is what most early-stage ATS/hiring tools do until they reach a scale that justifies pursuing a formal partnership.

Recommendation: keep the manual-share magic link as the real feature for this build window. Treat "native API push to LinkedIn/Naukri" as a Later-stage partnership goal, not an engineering task you can just build.

4.2 ATS integration

Reality: "ATS integration" is not one thing — every ATS (Workday, Greenhouse, Lever, Zoho Recruit, etc.) has a different API, different auth model, and some have none at all for outbound webhooks.

What's actually feasible now: a generic outbound webhook + CSV export of finalized candidate data (resume, transcript, scorecard) that any employer's ops team can wire into whatever they use. This is fast to build (a few days of engineering) and works regardless of which ATS the client has.

What's a real integration project (weeks, per ATS): native, authenticated, bidirectional sync with a specific ATS — pick one or two of your most-requested ATSs among actual pilot customers before building this, don't build generic multi-ATS support speculatively.

Important, from earlier in your own materials: never present ATS integration as a current feature in a pitch or on the website until it's actually shipped — the webhook/export version is real and fine to mention as "available," but a specific ATS's native sync should only be claimed once it exists.

4.3 Auto-publish to an employer's own career page

Reality: This is the one item that needs genuine re-scoping, not just more engineering time. There is no generic technical way to "write into" another company's website — every company's career page is built on different tech (Workday, Greenhouse-embedded, custom CMS, static HTML) with no common write API you can target. "If Amazon publishes a job, it appears on Amazon's career page automatically" is not something a third-party vendor can build without Amazon's own engineering team building the receiving end.

What's actually feasible and gets you 90% of the value:

- An **embeddable widget/iframe** (a script tag the employer's web team pastes into their own career page) that pulls live job listings from QuickRuit and renders them on the employer's site. This is standard, well-understood technology (many ATS/job-board products do exactly this) and is entirely within your control to build.
- A **public JSON API** of an employer's live jobs, so the employer's own dev team (if they want to) can pull and render it however they like.

Recommendation: rename this feature internally from "auto-push to career page" to "embeddable careers widget" — same underlying value for the employer, but it's something you can actually ship without depending on another company's engineering team.

5. Feasibility Ratings — Every Open Item

Item	Feasibility	Why
Job creation w/ module + duration selection	✓ Easy — already built	Pure internal CRUD/config, no external dependency
Magic link + DPDPA consent + resume upload	✓ Easy — already built	Standard flow
Candidate approval + LinkedIn profile link display	✓ Easy	Just store/display the URL the candidate provides — do not attempt to scrape LinkedIn profile data, their ToS blocks this
Interview invite via email	✓ Easy — already built	Standard transactional email
Module gating/progression (can't skip until done/time-up)	✓ Easy-Medium	Straightforward state machine on the frontend + backend validation
Tab-switch / copy-paste / face-presence proctoring	✓ Medium — largely built	MediaPipe-based, already referenced in your existing architecture
Question bank (aptitude + technical) at real scale	● Medium — needs content ops, not just engineering	The engineering (storage, delivery, difficulty tagging) is easy; the hard part is <i>content</i> — you need hundreds of quality-checked questions per role/domain. Fastest realistic path: LLM-generate a large draft bank, then have a human (you or a hired content reviewer) validate/edit before it goes live. Don't ship unreviewed AI-generated questions directly to candidates.
Coding assessment (DSA + take-home) with sandboxed execution	● Medium	Requires a code execution sandbox — use an existing service (e.g., Judge0, Piston) rather than building your own sandboxing/security layer from scratch; that's a solved problem, don't reinvent it
AI hints during coding (without giving the answer)	● Medium	Feasible via careful prompt design with explicit "never reveal the solution, only nudge" instructions and testing against attempts to jailbreak it into giving answers

Dynamic AI interviewer ("Recruiter Rian") with real follow-ups	 Medium-Hard	Technically doable with your existing Gemini-based setup, but real-time, low-latency, natural conversation (not a 2-5 second delay per turn) needs a genuine engineering push — streaming responses, possibly a faster-inference provider, and careful conversation-state management across modules. This is your single biggest and most valuable engineering investment — budget real time for it, don't treat it as a small feature
Unified scorecard combining all modules	 Medium	Primarily a backend aggregation + frontend design problem, not a hard research problem — you already generate scores per module, the work is combining them into one coherent view. This is high-value, relatively low-risk — prioritize it early
Behavioral analysis (tone, confidence signals)	 Medium, with a caution	Technically feasible with existing signal-processing/LLM approaches, but stay at the level of observable signals (pace, filler words, response structure) rather than claiming to infer psychological traits or "culture fit" — that's the exact territory that got HireVue into regulatory trouble (see your own competitive research on this)
Job board push (LinkedIn/Naukri native API)	 Hard — external dependency	Requires formal partner approval from LinkedIn/Naukri, not just engineering effort; not realistic inside a 3-month solo/small-team build
ATS integration (native, per-ATS)	 Medium per ATS, but pick your targets	Generic webhook/export is easy; native sync with a specific ATS is a real weeks-long project — do this only after a paying customer specifically needs a specific ATS
Auto-push to an employer's own career page (as originally framed)	 Not feasible as framed	No generic way to write into another company's website; re-scope to an embeddable widget (Section 4.3), which achieves the same practical outcome and is fully within your control
Screen-share detection (candidate sharing screen to another device/app)	 Not reliably feasible	Browsers do not expose an API to detect whether the user is screen-sharing their display to an external app/service. You can detect window/tab blur and, with permission, enumerate connected displays — but you cannot reliably detect "Zoom is running and sharing this screen to someone else." Set this expectation honestly rather than promising a security guarantee you can't deliver; document it as a known limitation, not a solved feature

6. Recommended Build Priority (Within the 3-Month Window)

Given the feasibility map above, sequence roughly like this:

1. **Unified scorecard** — highest value, lowest technical risk. This turns "we show marks per module" into the actual evidence-based product story you've locked as your positioning. Do this first.
 2. **Module gating + Start Interview landing page** — relatively small, high-polish, improves the candidate experience noticeably.
 3. **Coding sandbox (DSA + take-home) with AI hinting** — well-understood engineering, high perceived value for technical hiring.
 4. **Question bank buildout** — start now in parallel (it's a content/ops task, not blocked by other engineering), since it takes real calendar time regardless of when you start.
 5. **Dynamic AI interviewer (Recruiter Rian)** — start early because it's your biggest lift, but ship it incrementally: static-with-minor-follow-ups first, fully dynamic later. Don't block your 3-month launch on this being perfect.
 6. **Generic ATS webhook/export + embeddable careers widget** — build these lightweight versions; they cover most of the practical value of "integration" without chasing the unrealistic full auto-push version.
 7. **Behavioral analysis refinement + native per-ATS integrations** — defer to after you have paying pilot customers who specifically need them; don't build speculatively.
-

7. Open Decisions You Should Lock Before Building Further

- **Canonical module order** — you described the sequence inconsistently (technical→video→coding in one pass, coding-before-technical in another). Pick one default order, make it configurable per job, and document it once so engineering doesn't build against ambiguity.
- **What "Recruiter Rian" is allowed to say during coding** — define the hint boundary explicitly (e.g., "may point at the wrong data structure, may not write code") so this is a product spec, not left to prompt-tuning guesswork later.
- **Screen-share detection** — decide now whether to quietly drop this claim, or replace it with what you can actually deliver (tab/window focus tracking + face presence), so it doesn't end up over-promised in a pitch later.